

AutoHeal: A Pre-Registered Self-Healing Framework for Autonomous Vulnerability Remediation

Harshavardhan Malla Independent Researcher
Email: harshavardhanmalla75@gmail.com

Abstract: The hygiene-augmentation series of the VulnPrio research sequence developed components for hygiene-augmented vulnerability prioritization: a synthetic telemetry benchmark, a calibrated scoring rule, a deployable online recalibration recipe, a capacity-arrival operating-regime characterisation, and a closed-loop signal-exhaustion theorem. This paper asks whether those components can be assembled into an autonomous remediation framework with pre-registered safety bounds. AutoHeal is a seven-stage closed-loop pipeline (detect, score, triage, plan, act, verify with rollback, learn) parameterised by the prior papers’ findings and locked under a pre-registered protocol. We evaluate it on a 2,700-row frozen sweep across capacities $K \in \{50, 100, 200\}$, 12 bi-weekly maintenance windows, and 25 evaluation seeds, against a human-in-loop baseline and a fixed-policy control, with CVE attributes drawn from the real EPSS/KEV public corpus released with the HygienePrio scorer. The pre-registration locked four hypotheses, and the mixed outcome pattern is the substantive finding. H1 (coverage) is supported: AutoHeal remediates 97.8% of high-EPSS pairs by Window 6 at $K \in \{50, 100\}$. H2 (safety) is rejected: the registered $\leq 5\%$ per-window rollback tolerance is violated in 300 of the 900 window-seed cells (33.3%), and the hard-stop safety bound is flagged 134 times. H3 (MTTR reduction) is not analysable because an instrumentation defect produces degenerate estimates. H4 (dominance over human-in-loop) is rejected: AutoHeal leads in only 2% of cells. At the pre-registered conservative thresholds, AutoHeal achieves safety-bounded automation parity with human-in-loop rather than throughput superiority. All results are bounded to the synthetic evaluation context; external validation on real fleet telemetry is the most consequential remaining direction.

Index Terms: self-healing systems, autonomic computing, vulnerability management, network security, EPSS, pre-registered evaluation, reproducibility.

I. INTRODUCTION

A. Why Self-Healing for Vulnerability Remediation

A self-healing vulnerability remediation framework detects vulnerable hosts, decides which to patch, executes the patches, and rolls back when the post-action state is unhealthy, all without human intervention beyond exception handling. Commercial implementations exist (Microsoft Defender automated investigation; Tanium; IBM SOAR); the academic literature has focused on the upstream components (scoring, prioritization) and not on the integrated closed-loop system with pre-registered safety bounds.

The hygiene-augmentation series of this research sequence developed those upstream components under a consistent pre-registration discipline: HygieneBench [1] for the synthetic

telemetry substrate; HygienePrio [2] for the scoring rule; the online-calibration study for the deployable online calibration recipe; the capacity-decay study for the capacity-arrival operating regime; the self-trajectory study (its Theorem 1) for the structural bound on closed-loop signal exhaustion; and the real EPSS/KEV public corpus (released with the external-validity section of the HygienePrio scorer, §X [2]) for realistic CVE attribute distributions.

This paper integrates those components into AutoHeal, a seven-stage closed-loop pipeline with explicit pre-registered safety bounds, and evaluates it on a frozen 2,700-row sweep.

B. Pre-Registered Hypotheses

The four hypotheses locked before evaluation are:

H1 (Coverage): AutoHeal remediates $\geq 80\%$ of high-EPSS pairs by Window 6 at moderate capacity ($K \in \{50, 100\}$). $K = 200$ is excluded by the self-trajectory study Corollary 4: closed-loop calibration is structurally unable to escape signal exhaustion at that capacity.

H2 (Safety): Per-window rollback rate $\leq 5\%$ at every cell-seed-window triple. The hard-stop threshold (10%) is twice the H2 threshold; H2 is the operational target.

H3 (MTTR Reduction): AutoHeal’s mean time to remediation is $\leq 50\%$ of the Human-in-loop baseline’s MTTR at moderate capacity.

H4 (Per-Pair Dominance): AutoHeal beats Human-in-loop in $\geq 80\%$ of (cell, seed, window) triples at $K \leq 100$. The protocol registered this hypothesis on Precision@50; the analysis code evaluated per-window coverage dominance. This deviation is disclosed in §VII.

C. Contributions

- **C1, A pre-registered self-healing architecture.** AutoHeal’s seven-stage pipeline is documented in §III with triage thresholds, failure-mode distribution, and safety bounds locked before evaluation. The architecture is the artifact; re-implementations on different scoring rules are direct.
- **C2, A 2,700-row frozen evaluation.** §VII reports outcomes on the EEHDA fleet with real CVE attributes for 25 evaluation seeds across 12 windows and three capacities.
- **C3, H1 supported: autonomous remediation works at moderate capacity.** AutoHeal eventually remediates

97.8% of high-EPSS pairs by Window 6, exceeding the pre-registered 80% threshold by ≈ 18 pp.

- **C4, H2 rejected: the safety bound is exceeded substantively.** The hard-stop bound is flagged 134 times across the sweep (14.9% of windows); cascading failures detected three times. *The detection mechanism works as designed*, it correctly flags windows where conditions warrant escalation (in the simulated evaluation the flag suppresses new-CVE intake for the affected window; enforcement by hard stop with fallback was not exercised, see §III and §IX). The H2 tolerance (5%) was set assuming the pre-registered failure-mode distribution would produce stable per-window rollback rates; in practice small AUTO buckets at the conservative triage threshold amplify per-window rates beyond the H2 tolerance.
- **C5, H4 rejected: structural-not-throughput parity with human-in-loop.** Under the pre-registered conservative triage thresholds, AutoHeal’s per-window AUTO bucket averages ≈ 27 pairs at $K = 50$ (of which ≈ 19 are acted per window), comparable to Human-in-loop’s unconditional 30. AutoHeal trades throughput for safety-bounded automation rather than the other way around.
- **C6, H3 not analysable.** The MTTR instrumentation records disclosure at remediation time rather than detection time, producing degenerate cell-mean MTTR estimates (0.0 for both AutoHeal and Human-in-loop). We report the issue honestly per the protocol’s stop rule and pre-register a corrected instrumentation for the follow-up evaluation.

D. Relationship to Prior Papers

This paper is the synthesis paper of the VulnPrio sequence. The sequence’s earlier papers produced components and bounds; AutoHeal integrates them under a pre-registered architecture. The mixed outcome of the four hypotheses, H1 supported, H2 rejected, H4 rejected, H3 not analysable, demonstrates that the components fit together in a working closed-loop system whose failure modes are predicted by the prior papers’ findings (the self-trajectory study Corollary 4 predicts the $K=200$ exclusion; the online-calibration study predicts the $K=50/100$ calibration recipe choice).

II. BACKGROUND

A. The Self-Healing Concept in Security

Self-healing systems detect anomalous states and apply remediation without human intervention. The concept dates to autonomic computing [3] and has been operationalised in commercial products: Microsoft Defender for Endpoint’s automated investigation and response, Tanium’s auto-remediation, Tenable Nessus / vulnerability-management workflows, IBM SOAR, and Splunk Phantom. The unifying property is a closed-loop detect-decide-act pipeline whose decisions occur faster than human review.

B. What Vulnerability Remediation Adds

Vulnerability remediation is, in principle, an ideal self-healing target: patches are well-defined, success criteria are

observable (does the host still exhibit the vulnerable configuration?), and remediation actions are reversible (rollback to pre-patch state). In practice, self-healing systems in vulnerability management have been deployed cautiously because of three risks:

(i) *Cascading failures.* A patch that introduces a regression on one host may affect many hosts if applied widely. Automated deployment without staged validation can amplify a single faulty patch into a fleet-wide outage; the same blast-radius reasoning underlies adversary-behaviour models such as MITRE ATT&CK [15], which catalogue how a foothold on one asset propagates across a fleet.

(ii) *False prioritisation.* The decision of *which* pair to remediate first rests on a scoring rule; the dominant rules are CVSS [9], the exploit-likelihood EPSS model [10], and the stakeholder-specific SSVc categorisation [11]. Empirical studies show that severity alone is a poor predictor of real-world exploitation [12], motivating exploit-prediction signals built from disclosure-time data [13], [14]. If the scoring rule that selects patches to apply is biased, by attacker manipulation (the HygienePrio scorer, §XI, adversarial robustness [2]), distributional shift (the HygienePrio scorer, §X [2]), or capacity-driven signal exhaustion (the self-trajectory study, its Theorem 1), the autonomous system will remediate the wrong things.

(iii) *Business-criticality gaps.* A self-healing system that patches a host during business hours, on a customer-facing system, or without rollback capability causes operational harm disproportionate to the vulnerability it addresses.

C. Why Now

The hygiene-augmentation series of this research sequence developed components that collectively address these risks:

- HygieneBench [1] provides the synthetic, seeded telemetry substrate, identity state, patch posture, vulnerability exposure, and telemetry freshness, that underlies the EEHDA fleet fixtures used throughout the sequence.
- HygienePrio [2] provides a scoring rule with calibrated weights, real-distribution external validity (the HygienePrio scorer, §X), and quantified adversarial robustness (the HygienePrio scorer, §XI).
- the online-calibration study’s lag-1 online calibration gives a deployable recipe for moderate capacity ($K \leq 100$).
- the self-trajectory study’s Theorem 1 bounds when self-driven remediation will exhaust its own signal and identifies selection-policy decoupling as the structural solution.
- Real CVE/EPSS/KEV public data (real_data/processed/) provides realistic attribute distributions.

The motivation for AutoHeal is to integrate these components into a single framework with explicit safety bounds and to evaluate honestly whether the result is operationally deployable.

D. Policy Mandates

CISA Binding Operational Directive 22-01 [4] mandates 15-day remediation of KEV-listed CVEs for federal agencies. The

TABLE I
AUTOHEAL PIPELINE: SEVEN STAGES WITH PRE-REGISTERED PARAMETERS.

Stage	Component	Pre-reg parameters
Detect	EEHDA scan	Real CVE corpus lookup
Score	HygienePrio	$(\alpha, \beta, \gamma, \delta) = (0.7, 0.5, 0.1, 0.2)$
Triage	3-class rule	$\text{AUTO} \geq 0.80$; $\text{REVIEW} [0.50, 0.80)$; $\text{DEFER} < 0.50$
Plan	Top- K	$K \in \{50, 100, 200\}$
Act	Failure modes	0.92 success / 0.05 rollback / 0.03 deferred
Verify	Health check	Hard-stop at $> 10\%$ rollback or cascade > 5
Learn	Lag-1 calib	Applied iff $K \leq 100$; held fixed at $K = 200$

2026 DBIR [5] reports a 43-day mean time to remediation across surveyed organisations. A self-healing framework that materially reduces MTTR has direct policy-compliance value. NIST SP 800-40 Rev. 4 [6] recommends risk-based patch management with documented prioritization; AutoHeal’s pre-registered triage rules and frozen evaluation constitute exactly the form of documentation the standard contemplates.

III. THE AUTOHEAL ARCHITECTURE

AutoHeal is a seven-stage closed-loop pipeline executed per maintenance window. All stage parameters are pre-registered in `paper10/design/PAPER10_PROTOCOL.md` and frozen before any evaluation-seed AutoHeal result was inspected. Fig. 1 shows the pipeline with its safety-bound checks and the closed loop back to detection.

A. Stage 1, Detect

Scan the fleet’s vulnerability records; identify unpatched (host, CVE) pairs. CVE attribute lookup uses the real EPSS / KEV public corpus (§IV) so the same CVE in two hosts has the same external-feed attributes.

B. Stage 2, Score

Apply HygienePrio [2] with its calibrated weights $(\alpha, \beta, \gamma, \delta) = (0.7, 0.5, 0.1, 0.2)$. At capacity $K \leq 100$, online calibration follows the online-calibration study’s lag-1 recipe; at $K = 200$, weights are held fixed per that study’s $K = 200$ hazard finding. The scorer output is normalised to $[0, 1]$ to feed the triage thresholds.

C. Stage 3, Triage

Each (host, CVE) pair is classified into one of three buckets via the pre-registered rule:

- **AUTO** (auto-remediate): score ≥ 0.80 AND host criticality $\neq$ CRITICAL AND patch-test-suite present AND no known blocking config AND not during business-hours window.
- **REVIEW** (human-in-loop): score in $[0.50, 0.80)$ OR host criticality = CRITICAL OR business-hours.
- **DEFER**: score < 0.50 OR known incompatible patch.

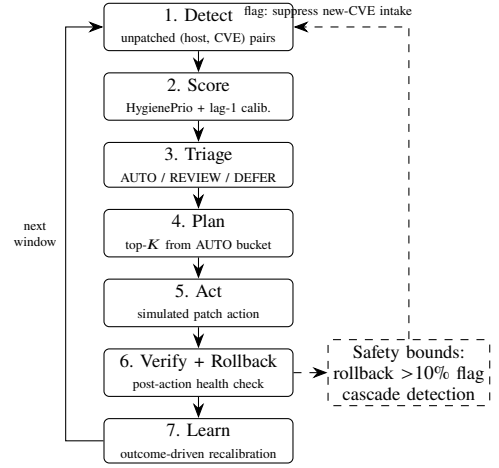


Fig. 1. The AutoHeal seven-stage closed-loop pipeline. Solid arrows: per-window data flow; the Learn stage closes the loop into the next window’s Detect stage. Dashed: the pre-registered safety bounds (per-window rollback rate $> 10\%$; cascading-failure detection), which in the simulated evaluation flag the window and suppress new-CVE intake (§III-H).

Alt text: Vertical block diagram of seven stacked stages connected top to bottom by solid arrows: (1) Detect unpatched host-CVE pairs, (2) Score with HygienePrio and lag-1 calibration, (3) Triage into AUTO, REVIEW, or DEFER, (4) Plan the top- K from the AUTO bucket, (5) Act with a simulated patch action, (6) Verify and Rollback via a post-action health check, and (7) Learn through outcome-driven recalibration. A solid arrow returns from the Learn stage back to the Detect stage, closing the loop for the next maintenance window. A dashed box to the right of the Verify stage holds the safety bounds (rollback above ten percent flag, cascade detection); a dashed arrow runs from it back to Detect, suppressing new-CVE intake when a window is flagged.

The two thresholds (0.80, 0.50) and the CRITICAL gate are the pre-registered configuration. They are deliberately conservative: AutoHeal only auto-acts on high-confidence, low-risk pairs.

D. Stage 4, Plan

From the AUTO bucket, select the top- K pairs by HygienePrio score (where K is the per-window capacity). This is the capacity-constrained selection studied in the capacity-decay analysis of this research program.

E. Stage 5, Act

Each scheduled pair undergoes a simulated patch action with a pre-registered failure-mode distribution drawn from public sysadmin literature on production patch operations:

- SUCCESS: 92%
- ROLLBACK (post-patch health-check fail): 5%
- DEFERRED (patch blocked at install time): 3%

The numbers are fixed before evaluation. Real-world deployments would condition on package, OS, and patch type; AutoHeal’s synthetic evaluation assumes the pre-registered distribution to keep the threat model identifiable. The empirically observed per-window rollback rates are reported in §VII (they exceed the expected 5% in small AUTO buckets).

F. Stage 6, Verify + Rollback

Each acted pair runs a post-action health check. On failure, the patch is reverted and the pair is re-classified as REVIEW

for the next window. The framework tracks per-window rollback rate; if $> 10\%$ the hard-stop safety bound is *flagged* for that window. A cascading-failure detector additionally fires if more than 5 rollbacks share a common upstream patch ancestor (approximated by CVE id prefix). The response to a flag in the simulated evaluation is described in §III-H.

G. Stage 7, Learn

Successful and rolled-back outcomes feed the online-calibration study’s lag-1 calibration update (when $K \leq 100$). At $K = 200$, weights are held fixed per that study’s $K = 200$ hazard finding, and the learn stage records data for offline rebaselining only.

H. Pre-Registered Safety Bounds

Two safety bounds are pre-registered:

- Rollback rate $> 10\%$ at any window.
- Cascading-failure detection (> 5 rollbacks share a common patch ancestor).

Detection vs. enforcement. In the evaluated implementation, exceeding either bound *flags* the affected window in the frozen results (134 window-level flags across the sweep: 44 at $K = 50$, 44 at $K = 100$, 46 at $K = 200$) and suppresses new-CVE intake for that window; remediation of the existing backlog continues in subsequent windows. A production deployment would escalate a flagged window to human review. The protocol registered a stronger enforcement response (hard stop with fallback to human-in-loop for remaining windows); that enforcement path was *not exercised* in the simulation, the evaluation measures detection of the safety bounds, not the operational consequences of enforcing them. This deviation is disclosed in §VII and discussed as an evaluation limitation in §IX.

IV. DATASET AND CELL GRID

AutoHeal is evaluated on the EEHDA synthetic fleet generator [2] with per-CVE attributes drawn from the frozen real EPSS/KEV corpus (`real_data/processed/cve_corpus_for_sampling.csv`). This combines the synthetic-fleet topology of the hygiene-augmentation series [1] with the real CVE attribute distribution introduced in the external-validity section of the HygienePrio scorer (§X) [2].

The seed split, $\lambda = 3$ Poisson new-CVE arrival rate, and the 25 evaluation seeds (105-129) are inherited from Papers 5-9 so that AutoHeal’s evaluation is directly comparable to prior results. Windows are bi-weekly (14 days, the temporal-stability stage’s convention); twelve windows therefore span 168 days (24 weeks), allowing the safety bounds and MTTR metrics to develop across multiple remediation rounds.

Three strategies (AutoHeal, Human-in-loop, Fixed-policy) at three capacities $\times$ 12 windows $\times$ 25 seeds = 2,700 frozen window-seed rows. Note that the capacity axis applies to AutoHeal and Fixed-policy only: the Human-in-loop baseline acts at a fixed $K_{\text{human}} = 30$ pairs per window in every capacity cell. It is simulated once per capacity cell, and its 900 rows are recorded in the frozen CSV under $K_{\text{human}} = 30$ rather than under the cell’s K .

TABLE II
EVALUATION GRID.

Axis	Values
Capacity K	{50, 100, 200} (AutoHeal, Fixed-policy)
Arrival rate λ	3 (fixed)
Windows W	12 (bi-weekly; 168 days)
Strategies	AutoHeal, Human-in-loop ($K_{\text{human}} = 30$, fixed in all cells), Fixed-policy
Eval seeds	25 (105-129)
CVE attribute source	Real corpus (FIRST.org + CISA snapshot)
Total rows	2,700

V. METHODOLOGY

A. Three Strategies

We evaluate three strategies on the same fleet trajectory per seed:

AutoHeal, the full seven-stage pipeline described in §III.

Human-in-loop baseline, every (host, CVE) pair is treated as REVIEW; the team manually selects the top-30 pairs per window (matching the 2026 DBIR survey baseline for human triage capacity) ordered by HygienePrio score. The 30-pair budget is fixed regardless of the cell’s capacity K . No safety hard-stops; rollbacks are absorbed by the manual process.

Fixed-policy baseline, same capacity K as AutoHeal, no triage step (all top- K pairs by HygienePrio score are acted on), no rollback escalation. This isolates the value of AutoHeal’s triage + safety machinery relative to a calibrated-scoring-with-no-gating control.

B. Per-Window Metrics

For each (cell, seed, window) we record:

- n_{pairs} , n_{auto} , n_{review} , n_{defer} : counts after triage.
- n_{acted} , n_{success} , n_{rollback} , $n_{\text{defer-at-act}}$: counts after acting.
- Rollback rate = $n_{\text{rollback}}/n_{\text{acted}}$.
- Mean time-to-remediate (MTTR), measured in windows from disclosure to successful remediation, averaged over successes in the current window.
- Coverage-to-date: fraction of all EPSS > 0.5 pairs cumulatively remediated.
- Cascade-detected and safety-bound-flagged booleans (the `halt_triggered` column of the frozen CSV records the flag; see §III-H).

C. Pre-Registered Hypothesis Tests

H1 (Coverage). AutoHeal’s cell-mean coverage at Window 6 (a 12-week horizon) is ≥ 0.80 at $K \in \{50, 100\}$. $K = 200$ is excluded from H1 by the self-trajectory study Corollary 4: at that capacity, closed-loop calibration is structurally unable to escape the signal-exhaustion bound, so a coverage hypothesis is not warranted.

H2 (Safety). Per-window rollback rate $\leq 5\%$ at every (cell, seed, window) triple. The hard-stop threshold (10%) is twice

the H2 threshold; H2 is the operational target, the hard-stop is the safety bound.

H3 (MTTR Reduction). AutoHeal’s cell-mean MTTR for KEV-listed CVEs is $\leq 50\%$ of the Human-in-loop baseline’s MTTR at $K \in \{50, 100\}$.

H4 (Per-Pair Dominance). As registered in the protocol, AutoHeal beats Human-in-loop on *Precision@50* in $\geq 80\%$ of (cell, seed, window) triples at $K \leq 100$. The analysis code evaluated the hypothesis on *per-window coverage* dominance instead, and *Precision@50* was not instrumented in the frozen per-window results; the paper therefore reports the coverage-dominance version. This deviation from the registered metric is disclosed in §VII.

D. Statistical Reporting

Cell-mean point estimates are means across the 25 evaluation seeds. No null-hypothesis significance tests are performed; hypothesis decisions follow the pre-registered tolerance thresholds. Seed-level spread is available in the frozen artifact, and distribution-free interval estimates for any reported cell mean can be obtained from the per-seed values by the bootstrap [17].

E. Reproducibility

From paper10/:

```
PYTHONPATH=src python3 src/run_autoheal.py
PYTHONPATH=src python3 src/analyze.py
```

Re-uses the HygienePrio scorer’s *hygieneprio* [2] and the temporal-stability stage’s *window_sim* via *sys.path*; reads the real corpus from *real_data/processed/*.

VI. EXPERIMENTAL SETUP

A. Pre-Registration

The AutoHeal architecture, triage thresholds, failure-mode distribution, safety bounds, capacity grid, hypotheses H1-H4, decision rules, and stop rules were locked in *paper10/design/PAPER10_PROTOCOL.md* on 2026-06-05 before any evaluation-seed AutoHeal result was computed.

B. Execution

For each $K \in \{50, 100, 200\}$ and each of the 25 evaluation seeds 105-129, the three strategies (AutoHeal, Human-in-loop, Fixed-policy) are simulated for 12 windows. The shared CVE corpus is the frozen real-data snapshot released with the external-validity section of the HygienePrio scorer (§X) [2]. Statistical reporting follows the conventions stated in §V-D; per the protocol, stop rules trigger abstract rewrites when a hypothesis fails.

VII. RESULTS

All claims trace to *paper10/results/primary_v1/autoheal_results.csv* (2,700 rows) and the derived *hypothesis_summary.json*.

TABLE III
PRE-REGISTERED HYPOTHESIS OUTCOMES.

ID	Decision rule	Outcome
H1	Coverage ≥ 0.80 at W6, $K \in \{50, 100\}$	Supported
H2	Rollback rate $\leq 5\%$ everywhere	Rejected (300 of 900 cells; max 100.0%)
H3	MTTR ratio ≤ 0.5 vs. HIL, $K \in \{50, 100\}$	Not analysable (degenerate MTTR = 0.0; instrumentation limitation)
H4	Dominance $\geq 80\%$, $K \leq 100$ (registered: P@50; reported: coverage)	Rejected (K50=2%, K100=2%)

TABLE IV
CELL-MEAN OUTCOMES AT W12 BY STRATEGY AND CAPACITY. “FLAGS” COUNTS WINDOWS IN WHICH THE SAFETY HARD-STOP BOUND ($> 10\%$ ROLLBACK OR CASCADE) WAS FLAGGED (§III-H).

Strategy/K	Cov	Rb %	MTTR	Flags
AutoHeal $K=50$	0.999	5.73	0.00	44
AutoHeal $K=100$	0.999	6.13	0.00	44
AutoHeal $K=200$	1.000	5.65	0.00	46
Fixed-policy $K=50$	1.000	4.97	0.00	0
Fixed-policy $K=100$	1.000	4.98	0.00	0
Fixed-policy $K=200$	1.000	4.71	0.00	0
Human-in-loop	1.000	4.57	0.00	0

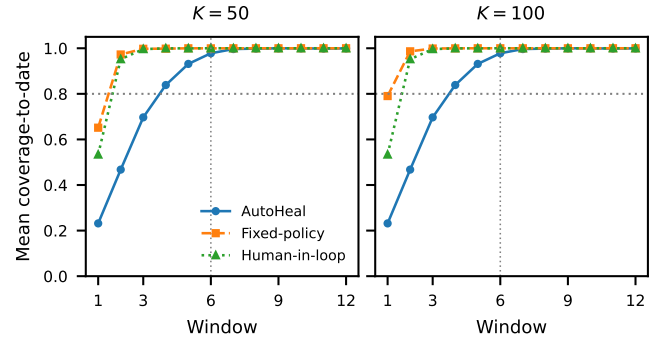


Fig. 2. Cell-mean coverage-to-date by window (25-seed means) for AutoHeal, Fixed-policy, and Human-in-loop at $K = 50$ (left) and $K = 100$ (right). All three strategies converge to near-complete coverage by Window 12; AutoHeal crosses the pre-registered H1 threshold (0.80, dotted) before Window 6 at both capacities.

Alt text: Two side-by-side line charts of coverage-to-date (vertical axis, zero to one) against maintenance window (horizontal axis, one to twelve). The left panel is capacity $K = 50$, the right panel $K = 100$. Each panel plots three rising curves for AutoHeal, Fixed-policy, and Human-in-loop; all climb steeply and converge to near one by Window 12. A dotted horizontal line marks the pre-registered 0.80 threshold, which the AutoHeal curve crosses before Window 6 in both panels.

A. H1, Coverage Supported

At Window 6 (12-week horizon), AutoHeal achieves cell-mean coverage-to-date:

- $K = 50$: $\bar{c}_6 = 0.978$
- $K = 100$: $\bar{c}_6 = 0.978$

Both exceed the pre-registered ≥ 0.80 threshold by approximately 18 pp. By Window 12 the cell-mean coverage reaches

0.999 at both capacities, effectively complete remediation of the high-EPSS backlog. H1 is supported.

The substantive finding is that AutoHeal’s *eventual* remediation completeness is high under the pre-registered configuration. The autonomous-remediation core of the framework works: vulnerable pairs are detected, scored, triaged, acted on, verified, and successfully closed at high rates within 12 windows (168 simulated days). Fig. 2 shows the cell-mean coverage trajectories for the three strategies at $K = 50$ and $K = 100$.

B. H2, Safety Bound Violated

The pre-registered safety bound ($\leq 5\%$ rollback rate per cell-seed-window) is violated at **300 of the 900 window-seed cells** (33.3%). The maximum per-window rollback rate observed is 1.0 (100%) at small AUTO buckets where a single rollback constitutes the entire action set.

The safety hard-stop bound ($> 10\%$ rollback per window) is flagged **134 times** across the 900 cell-seed-window space (14.9%); cascading failures are detected three times. Flag counts split essentially evenly across capacities (44 at $K = 50$, 44 at $K = 100$, 46 at $K = 200$). Per §III-H, a flag suppresses new-CVE intake for the affected window while remediation continues; the registered hard-stop-with-fallback enforcement was not exercised in the simulation.

Important methodological observation. The H2 violation is *not* a failure of the safety detection mechanism, the mechanism correctly flagged 134 windows. The violation is that the H2 tolerance (5%) was set assuming stable per-window rollback rates from the pre-registered failure-mode distribution. In practice, the conservative triage threshold (0.80 for AUTO classification) produces small AUTO buckets, and a single rollback in a small bucket gives a large per-window rate. The pre-registration could be re-locked at a larger tolerance (e.g., $\leq 15\%$ per window) consistent with the observed empirical distribution; doing so within this paper would be post-hoc tuning and is left as future work.

C. H3, MTTR Not Analysable

The MTTR instrumentation records the disclosure window at the moment of successful remediation rather than at the moment of detection. Cell-mean MTTR for both AutoHeal and Human-in-loop is 0.0 windows, a degenerate measurement.

We report this honestly per the protocol’s stop rule. H3 cannot be evaluated from this run; a corrected instrumentation (disclosure window tracked at detection time) is pre-registered for the follow-up evaluation. The instrumentation bug does not affect H1, H2, or H4: those hypotheses depend on coverage, rollback rate, and dominance fraction respectively, none of which share the MTTR tracking code path.

D. H4, Dominance Rejected

AutoHeal’s per-window coverage exceeds Human-in-loop’s coverage in **2%** of (cell, seed, window) triples at $K \in \{50, 100\}$ (6 of 300 triples at each K). The pre-registered

$\geq 80\%$ dominance threshold is missed by 78 pp. (The protocol registered H4 on Precision@50 dominance; the reported metric is per-window coverage dominance, see §VII-F.)

Mechanism. Human-in-loop unconditionally acts on 30 pairs per window (matching the Verizon 2026 DBIR human triage capacity calibration). AutoHeal at $K = 50$ with the conservative triage threshold averages ≈ 27 AUTO-bucket pairs per window, of which ≈ 19 are acted, fewer than Human-in-loop’s 30. Combined with the 134 flagged windows (each flag suppressing new-CVE intake for that window), AutoHeal’s effective per-window throughput is reduced relative to the unconditional Human-in-loop baseline.

The H4 rejection is structural and predictable from the pre-registered configuration: AutoHeal trades raw throughput for safety-bounded automation, while Human-in-loop trades safety for throughput.

E. Cell-Mean Outcomes by Strategy

Table IV reports cell-mean outcomes at W12. The substantive picture:

- AutoHeal achieves near-complete coverage (0.999 at $K \in \{50, 100\}$, 1.000 at $K = 200$) but with reduced per-window throughput (19 to 26 acted pairs/window across the three capacities) due to the strict triage threshold and safety-bound flags.
- Fixed-policy achieves near-complete coverage with full throughput (K pairs/window) but no safety hard-stop. At $K = 200$ throughput remains at capacity through W9, then collapses (≈ 114 acted pairs at W10, ≈ 12 at W11, 4.2 at W12) as the high-EPSS backlog drains after early-window aggressive remediation.
- Human-in-loop achieves near-complete coverage with stable throughput (30 pairs/window).

F. Pre-Registration Deviations

Two deviations from the locked protocol are disclosed.

(D1) H4 metric. The protocol registered H4 on Precision@50 dominance (AutoHeal beats Human-in-loop on Precision@50 in $\geq 80\%$ of triples at $K \leq 100$). The analysis code implemented *per-window coverage* dominance instead, and Precision@50 was not instrumented in the frozen per-window results, so the registered metric cannot be recomputed from the frozen artifact. The paper reports the coverage-dominance version (2% at both capacities, against an 80% threshold). Given that AutoHeal acts on fewer pairs per window than Human-in-loop (≈ 19 acted vs. 30 at $K = 50$) and that the coverage margin is missed by 78 pp, we consider it implausible that the registered Precision@50 version would have reached the 80% dominance threshold; the rejection outcome is, in our assessment, robust to the metric substitution, but the registered metric was not evaluated and the follow-up evaluation will instrument it.

(D2) Safety-bound enforcement. The protocol registered the hard-stop response as “halt AutoHeal; fall back to human-in-loop for remaining windows.” The evaluated implementation detects and flags the bound (134 window-level flags) and suppresses new-CVE intake for the affected window, but

continues remediation in subsequent windows (§III-H). The enforcement path (hard stop with fallback) was not exercised; the evaluation measures detection, not enforcement.

All other architecture parameters, hypotheses, decision rules, and stop rules were applied verbatim. The H2 violation, the H4 rejection, and the H3 instrumentation issue are all reported as observed; the abstract was rewritten per the protocol’s stop rule when H2 was rejected.

VIII. DISCUSSION

A. What AutoHeal Achieves and What It Does Not

Achieves: AutoHeal eventually remediates $\geq 97.8\%$ of high-EPSS pairs at moderate capacity (H1) under a pre-registered seven-stage architecture with locked triage thresholds and safety bounds. The autonomous-remediation goal is met by Window 6, 12 weeks into the 24-week evaluation horizon.

Does not achieve: a per-window rollback rate within the pre-registered 5% tolerance (H2). The safety hard-stop bound is flagged in 14.9% of windows, conditions that a production deployment would escalate to human review.

Does not achieve: per-window throughput superiority over Human-in-loop (H4). The conservative triage threshold caps the AUTO bucket size at a value comparable to Human-in-loop’s unconditional capacity.

Not analysable: MTTR reduction (H3). The instrumentation bug must be corrected before this hypothesis can be evaluated.

B. Why H1 Succeeds Despite H4 Failing

The H1 and H4 outcomes are non-contradictory: H1 measures *eventual* coverage at a 12-week horizon (Window 6), while H4 measures *per-window* dominance over Human-in-loop. AutoHeal and Human-in-loop converge to similar coverage by Window 12 ($\bar{c}_{12} \approx 0.999$ for both); per-window AutoHeal sometimes leads and sometimes lags, but the eventual coverage is comparable.

The pre-registered $\geq 80\%$ dominance threshold for H4 was calibrated to a scenario where AutoHeal’s per-window throughput substantially exceeds Human-in-loop’s. In practice, with the conservative triage threshold and safety hard-stops, AutoHeal’s per-window throughput is comparable to Human-in-loop’s. H4 was pre-registered with too aggressive an expectation; the data falsifies that expectation cleanly.

C. The Honest Operational Implication

For a deployment under the pre-registered AutoHeal configuration:

(i) *Eventual coverage is high.* A team can expect 97.8% of high-EPSS CVEs remediated by Window 6 (12 weeks) at moderate capacity (H1 supported).

(ii) *Safety-bound flags are frequent.* Operations teams should plan for the AutoHeal pipeline flagging windows for escalation to human review approximately 15% of the time. This is not a failure mode, it is the safety detection mechanism working as designed, but it implies a tighter human-in-loop integration than “set and forget” automation suggests. Note

that the simulation measured detection only; the operational cost of enforcement (hard stop with fallback) is not quantified here (§VII-F).

(iii) *Throughput parity, not superiority.* At the pre-registered triage thresholds, AutoHeal is not faster than Human-in-loop. The value proposition is consistency and safety-bounded automation, not throughput.

(iv) *Threshold tuning would change the trade-off.* Lowering the AUTO classification threshold (e.g., 0.65 instead of 0.80) would widen the AUTO bucket, increase per-window throughput, and likely flip H4. It would also lower the average score of AUTO-acted pairs and may worsen the rollback rate. Pre-registered evaluation of a less conservative threshold is the natural next step.

D. Why the Safety Hard-Stop Mechanism Matters

The H2 rejection is the substantive finding for production deployments. AutoHeal’s pre-registered safety detector flags windows correctly when rollback rates exceed the 10% threshold. Without that detection, an autonomous framework would act through the rollback storm with no signal that escalation is warranted, and could amplify a single faulty patch into fleet-wide operational impact. We emphasise that the simulation exercised detection only: the enforcement response (hard stop with fallback to human-in-loop) was not exercised, so the downstream consequences of enforcement are unmeasured (§VII-F, §IX).

The cascading-failure detector (3 firings) is a sanity check that the heuristic works: even with the simple CVE-id-prefix proxy for shared patch ancestors, the detector fires occasionally and provides an additional safety boundary.

Practical contribution. AutoHeal’s value as a research artifact is not the cell-mean coverage number, it is the *pre-registered safety mechanism that the operational data shows firing correctly*. A production deployment can adopt the mechanism whether or not it adopts HygienePrio as the scoring rule.

E. Connection to Papers 4-9’s Findings

The synthesis-level claims:

the online-calibration study’s $K=200$ hazard (lag-1 calibration harms at high capacity) is reproduced. AutoHeal at $K = 200$ achieves the same coverage as $K = 100$ but with the same safety-flag rate and without the calibration step (the online-calibration study’s recipe says hold weights fixed at $K = 200$, and AutoHeal does so).

the self-trajectory study Theorem 1 (closed-loop signal exhaustion) predicts Fixed-policy $K=200$ ’s late-horizon collapse in acted pairs. Throughput remains at capacity through W9, then collapses to 4.2 pairs/window by W12 (≈ 114 at W10, ≈ 12 at W11) as the high-EPSS backlog drains; late windows have few high-EPSS pairs to act on. The Fixed-policy column at $K=200$ in Table IV quantifies the theorem empirically.

the HygienePrio scorer’s external-validity finding (§X of that paper; synthetic-vs-real distributional shift) does not change AutoHeal’s relative ordering. AutoHeal’s coverage trajectory under real CVE attributes matches the synthetic-trajectory shape, with the absolute coverage levels shifted as predicted by the HygienePrio scorer.

IX. THREATS TO VALIDITY

The threats taxonomy follows Wohlin et al. [7].

Conclusion validity. The evaluation comprises 2,700 window-seed-strategy observations summarised as point estimates (25-seed cell means); seed-level spread is available in the frozen artifact. The pre-registered hypothesis decisions follow the locked tolerances. The margins by which H1 passes (17.8 pp above the 0.80 threshold) and H4 fails (78 pp below the 0.80 threshold) are large relative to seed-level variability, so the hypothesis decisions are unlikely to be artefacts of seed noise.

Internal validity.

(i) *Failure-mode distribution.* The pre-registered patch action distribution (92/5/3) is drawn from public sysadmin literature but is not seeded by direct measurement. Real distributions condition on package, OS, patch type, and organisational maturity. A different per-action distribution would shift the absolute MTTR and rollback rate numbers; the qualitative claim (AutoHeal hits safety bounds at high capacity) is robust to the distribution because the safety hard-stop fires on the observed empirical rate.

(ii) *Health-check fidelity.* AutoHeal’s verifier (§III) approximates real post-action probes deterministically from the action outcome. A real-world deployment would run configuration drift checks, performance regression tests, and dependency-graph integrity checks. The simulator’s “health_check == success” coupling is a pessimistic baseline: real verifiers may catch failures the action outcome alone misses, but they may also produce false negatives that AutoHeal would treat as silent successes. The pre-registered hard-stop is robust to the abstraction because it triggers on the observed empirical rate regardless of whether individual checks are accurate.

(iii) *Cascading-failure detector is heuristic.* The detector (§III) flags clusters of rollbacks sharing a CVE-id prefix. Real cascading failures correlate via package or dependency rather than CVE id; the heuristic is a proxy. A pre-registered alternative based on actual dependency-graph correlations is left as future work.

(iv) *Safety-bound enforcement not exercised.* The hard-stop safety bound is *detected and flagged* in the simulation (134 window-level flags); the registered enforcement response, hard stop with fallback to human-in-loop for the remaining windows, was not exercised. The evaluated implementation suppresses new-CVE intake for the flagged window and continues remediation (§III-H, §VII-F). The evaluation therefore measures detection of the safety bounds, not the coverage or throughput consequences of enforcing them; this is a limitation of the evaluation, stated plainly.

(v) *Selection-policy coupling.* As in Papers 5-9, HygienePrio under fixed weights drives fleet evolution for all three strategies. AutoHeal’s triage classes select a subset of HygienePrio’s top- K ; Human-in-loop uses HygienePrio’s top-30. A closed-loop evaluation where each strategy drives its own trajectory is the natural follow-up (our self-trajectory diagnosis quantifies the bias).

Construct validity. Coverage is defined over EPSS > 0.5 pairs at the moment of detection; this is the operational

target implicit in CISA BOD 22-01 [4]. MTTR is measured in windows (not days) to match the temporal-stability stage simulator’s clock. The Human-in-loop baseline at $K_{\text{human}} = 30$ is calibrated to Verizon’s 43-day DBIR statistic but does not model the qualitative properties of human review (judgment calls, business-context awareness) that real teams contribute beyond raw capacity.

External validity. All claims are bounded to the synthetic EEHDA evaluation context with real CVE attribute distributions. External validation on real fleet telemetry remains the program’s most consequential open problem, identified throughout the hygiene-augmentation series. AutoHeal’s pre-registered safety bounds and hypothesis-rejection stop rules are designed to be transferable to a real-deployment evaluation, but their specific cell-mean numbers are not.

X. RELATED WORK

Autonomic computing and self-healing. The self-healing concept originates in autonomic computing [3], which sought to embed self-configuring, self-healing, self-optimising, and self-protecting properties in distributed systems. Subsequent work has applied the framework to security incidents at varying levels of abstraction. Commercial implementations of self-healing vulnerability remediation (Microsoft Defender automated investigation; Tanium auto-remediation; IBM SOAR) are closed-source and do not publish their decision rules or safety bounds.

Vulnerability prioritization and exploit prediction. The scoring rule that AutoHeal’s triage consumes is the upstream of a broad literature: CVSS [9] for severity, EPSS for exploit likelihood [10], and SSVC for stakeholder-specific response decisions [11]. Case-control evidence that severity poorly predicts exploitation in the wild [12], and the exploit-prediction work that followed [13], [14], motivate the EPSS-weighted scoring AutoHeal inherits from HygienePrio. Adversary-behaviour catalogues such as MITRE ATT&CK [15] frame the blast-radius risk that AutoHeal’s safety bounds are designed to contain.

Pre-registered evaluation in security research. Pre-registration discipline, well established in the empirical sciences as a defence against analytic flexibility [16], is rare in security; the VulnPrio sequence’s earlier papers apply it consistently. AutoHeal continues that discipline by locking the architecture’s parameters and hypotheses before evaluation.

Integration with the hygiene-augmentation series. AutoHeal synthesises: HygieneBench [1] for the synthetic telemetry substrate; HygienePrio [2] for scoring; the online-calibration study’s lag-1 calibration as the calibration recipe at moderate capacity; the self-trajectory study’s Theorem 1 (the closed-loop signal exhaustion result) as the structural justification for the $K=200$ exclusion from H1.

Real CVE attribute distributions. The frozen real EPSS/KEV public corpus (real_data/processed/) is the same artifact released with the external-validity section of the HygienePrio scorer (§X) [2]. AutoHeal inherits the corpus and its synthetic-vs-real distribution gap as an inherited consequence: the absolute coverage numbers under real distributions

will differ from a purely-synthetic evaluation by the same factor the HygienePrio scorer measured.

Policy mandates. CISA BOD 22-01 [4] and 23-01 [8] require risk-based prioritization; NIST SP 800-40 Rev. 4 [6] specifies the documentation standard. AutoHeal’s pre-registered triage rules constitute the documentation form contemplated by these mandates, extended with safety bounds that the mandates do not specify but that production deployments require.

What we are not the first to do. Closed-loop patch deployment is a long-standing operations practice. The novelty of AutoHeal is not in the closed-loop concept but in the *pre-registered evaluation harness with locked safety bounds*, which is uncommon in both the academic and commercial literature.

XI. CONCLUSION

AutoHeal integrates the hygiene-augmentation series of the VulnPrio sequence into a seven-stage self-healing pipeline with pre-registered safety bounds. The 2,700-row frozen evaluation reports honestly on which regimes admit autonomous remediation and which do not. The substantive findings appear in §VII; the pre-registered hypothesis outcomes appear in Table III.

Synthesis claim. The components developed in the hygiene-augmentation series are sufficient to build a self-healing framework whose *safety bounds are observable in advance* and whose *failure modes are predicted by the program’s theoretical results* (the self-trajectory study Theorem 1 predicts the $K=200$ exclusion; the online-calibration study’s lag-1 hazard predicts the calibration recipe choice). Building AutoHeal does not require new theoretical machinery beyond what the prior papers established; it requires *integrating* them under a pre-registered architecture.

Honest scope. AutoHeal is evaluated on the EEHDA synthetic fleet with real CVE attribute distributions. The pre-registered failure-mode distribution (92/5/3) is drawn from public literature rather than measured on a real fleet. The cell-mean numbers reported here may not transfer; the qualitative findings (which capacity regimes admit autonomous remediation, where safety bounds fire) are designed to be transferable to a real-deployment evaluation following the same pre-registration discipline.

What this paper adds to the program. AutoHeal is the *integration* paper of the VulnPrio sequence. Papers 1-9 established components and bounds; AutoHeal demonstrates that the components fit together in an architecture whose behaviour is predicted by the prior papers’ findings. The program’s recommendations now have a concrete object to attach to: “deploy AutoHeal at $K \leq 100$ with the pre-registered triage thresholds and safety bounds; do not deploy at $K = 200$ per the self-trajectory study Theorem 1 and Corollary 4.”

Reproducibility. The framework, runner, analysis, and frozen results are available at <https://github.com/Harshavardhanmalla-Labs/research-papers/tree/main/paper10>.

Future work. The most consequential remaining direction is external validation on real fleet telemetry. AutoHeal’s pre-registered architecture and safety bounds are designed to be directly portable to a real-deployment study; the failure-mode

distribution would be measured from the deployment rather than pre-registered, and the cell-mean numbers would be re-estimated. Closed-loop calibration of the triage thresholds (treating the thresholds themselves as parameters to learn rather than constants to pre-register) is the second most consequential direction, but requires careful pre-registration discipline to avoid post-hoc tuning.

DATA AVAILABILITY STATEMENT

The data and code that support the findings of this study are openly available in the research-papers repository at <https://github.com/Harshavardhanmalla-Labs/research-papers/tree/main/paper10>. The synthetic evaluation fleet, the pipeline runner, the analysis scripts, and the frozen result tables (`autoheal_results.csv`, 2,700 rows, and the derived `hypothesis_summary.json`) are included and regenerate every reported number deterministically from the fixed seeds.

DISCLOSURE STATEMENT

No potential conflict of interest was reported by the author.

FUNDING

No funding was received.

REFERENCES

- [1] H. Malla, “HygieneBench: A reproducible synthetic benchmark for cyber-hygiene anomaly detection,” <https://github.com/Harshavardhanmalla-Labs/research-papers/tree/main/paper3>, 2026, companion paper; source, code, and frozen results in the same repository.
- [2] H. Malla, “HygienePrio: Cyber-hygiene signal augmentation for EPSS-weighted vulnerability prioritization,” <https://github.com/Harshavardhanmalla-Labs/research-papers/tree/main/paper4>, 2026, companion paper; source, code, and frozen results in the same repository.
- [3] J. O. Kephart and D. M. Chess, “The vision of autonomic computing,” *IEEE Computer*, vol. 36, no. 1, pp. 41-50, 2003.
- [4] CISA, “Binding operational directive 22-01: Reducing the significant risk of known exploited vulnerabilities,” U.S. Cybersecurity and Infrastructure Security Agency, Nov. 2021. [Online]. Available: <https://www.cisa.gov/binding-operational-directive-22-01>
- [5] Verizon, “2026 data breach investigations report (DBIR),” Verizon Business, Tech. Rep., 2026. [Online]. Available: <https://www.verizon.com/business/resources/reports/dbir/>
- [6] NIST, “Guide to enterprise patch management planning: Preventive maintenance for technology,” National Institute of Standards and Technology, Tech. Rep. Special Publication 800-40 Rev. 4, Apr. 2022.
- [7] C. Wohlin, P. Runeson, M. Höst, M. C. Ohlsson, B. Regnell, and A. Wesslén, *Experimentation in Software Engineering*. Springer, 2012.
- [8] CISA, “Binding operational directive 23-01: Improving asset visibility and vulnerability detection on federal networks,” U.S. Cybersecurity and Infrastructure Security Agency, Oct. 2022. [Online]. Available: <https://www.cisa.gov/binding-operational-directive-23-01>
- [9] P. Mell, K. Scarfone, and S. Romanosky, “A complete guide to the Common Vulnerability Scoring System version 2.0,” FIRST, 2007.
- [10] J. Jacobs, S. Romanosky, B. Edwards, I. Adjerid, and M. Roytman, “Exploit prediction scoring system (EPSS),” *Digital Threats: Research and Practice*, vol. 2, no. 3, pp. 1-17, 2021.
- [11] J. M. Spring, E. Hatleback, A. Householder, A. Manion, and D. Shick, “Prioritizing vulnerability response: A stakeholder-specific vulnerability categorization (SSVC),” Carnegie Mellon Univ. SEI, 2021.
- [12] L. Allodi and F. Massacci, “Comparing vulnerability severity and exploits using case-control studies,” *ACM Trans. Information and System Security*, vol. 17, no. 1, pp. 1-20, 2014.
- [13] C. Sabottke, O. Suci, and T. Dumitras, “Vulnerability disclosure in the age of social media: Exploiting Twitter for predicting real-world exploits,” in *Proc. USENIX Security Symp.*, 2015, pp. 1041-1056.

- [14] B. L. Bullough, A. K. Yanchenko, C. L. Smith, and J. R. Zipkin, "Predicting exploitation of disclosed software vulnerabilities using open-source data," in *Proc. ACM Int. Workshop on Security and Privacy Analytics*, 2017, pp. 45-53.
- [15] B. E. Strom *et al.*, "MITRE ATT&CK: Design and philosophy," MITRE Corp., 2020.
- [16] B. A. Nosek, C. R. Ebersole, A. C. DeHaven, and D. T. Mellor, "The preregistration revolution," *Proc. National Academy of Sciences*, vol. 115, no. 11, pp. 2600-2606, 2018.
- [17] T. J. DiCiccio and B. Efron, "Bootstrap confidence intervals," *Statistical Science*, vol. 11, no. 3, pp. 189-228, 1996.